

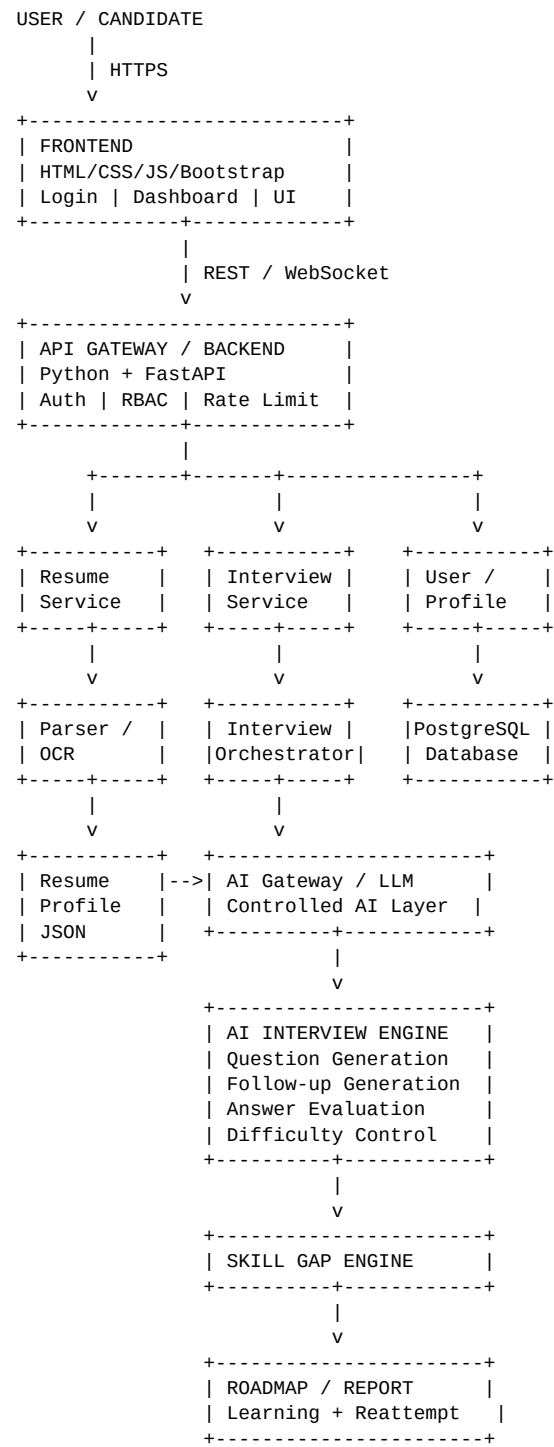
SYSTEM ARCHITECTURE DOCUMENT

AI Resume-Based Interview & Skill Gap Platform

Version: 1.0 Architecture Type: Production-oriented Web + AI Architecture

1. Architecture Overview

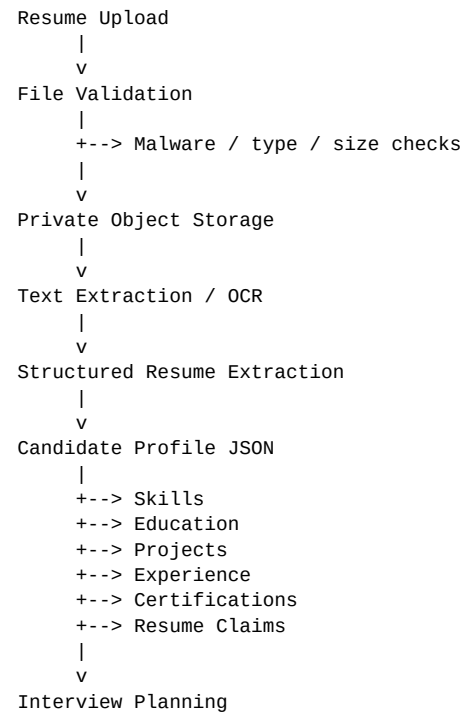
The platform is designed as a modular, API-first system. The frontend handles candidate interaction, the Python backend orchestrates business logic, specialised AI services handle resume analysis and interview intelligence, PostgreSQL stores structured application data, private object storage stores uploaded documents, and Redis/background workers handle caching and long-running jobs.



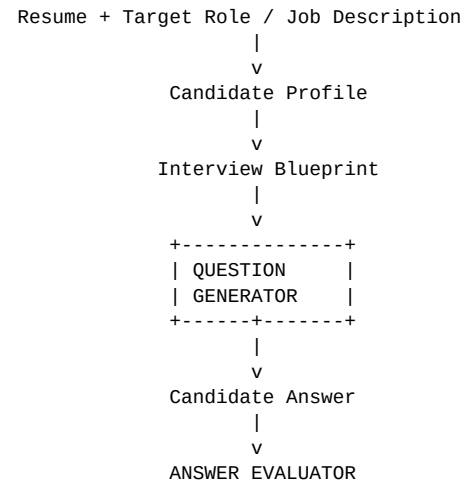
2. Major Components

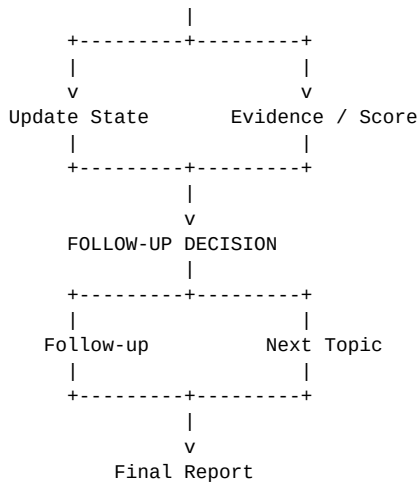
Component	Responsibility
Frontend	Resume upload, dashboard, interview screen, results and progress UI.
API / Backend	Authentication, authorisation, orchestration, validation and business rules.
Resume Service	Extract text and structured candidate information from PDF/DOC/DOCX.
Interview Service	Create interview blueprint, manage sessions and question flow.
AI Gateway	Centralise model access, prompts, structured outputs, retries and usage controls.
AI Interview Engine	Generate questions, evaluate answers, adapt difficulty and create follow-ups.
Skill Gap Engine	Compare required, claimed and demonstrated competencies.
Roadmap Engine	Convert gaps into prioritised learning and practice actions.
PostgreSQL	Persistent relational data.
Object Storage	Private storage for resumes and generated documents.
Redis / Workers	Caching, queues and asynchronous processing.

3. Resume Processing Flow



4. Interview Runtime Architecture





5. AI Service Decomposition

Do not build the entire product around one giant prompt. Each AI capability should have a defined input/output contract, validation and test cases.

- **Resume Extractor:** converts unstructured resume text into validated structured JSON.
- **Job Description Analyzer:** extracts required skills, responsibilities and seniority signals.
- **Interview Planner:** creates a competency blueprint and topic distribution.
- **Question Generator:** produces role- and resume-specific questions.
- **Follow-up Generator:** uses the candidate's answer and interview state to generate targeted follow-ups.
- **Answer Evaluator:** applies a structured rubric and produces evidence-backed scores.
- **Skill Gap Analyzer:** compares required, claimed and demonstrated competency levels.
- **Roadmap Generator:** converts priority gaps into learning/practice actions.
- **Report Generator:** creates the final candidate-readable assessment.

6. Data Architecture

```

PostgreSQL
|
+-- users
+-- candidate_profiles
+-- resumes
+-- skills
+-- projects
+-- experiences
+-- job_roles
+-- job_descriptions
+-- interviews
+-- questions
+-- answers
+-- evaluations
+-- skill_gaps
+-- roadmaps
+-- reports
+-- subscriptions
+-- audit_logs
  
```

```

Object Storage
|
+-- original resumes
+-- processed documents
+-- generated reports
  
```

```

Redis
|
+-- cache
  
```

```

+-- rate-limit state
+-- interview/session state where appropriate
+-- background job queues

```

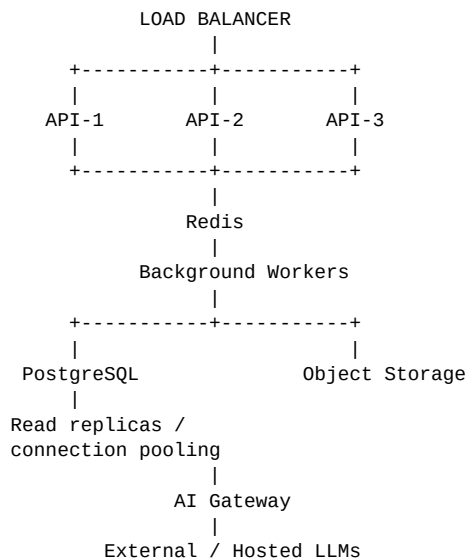
7. Security Architecture

- TLS/HTTPS for all external traffic.
- Secure password hashing and short-lived authenticated sessions/tokens.
- Role-Based Access Control (RBAC) for candidate, recruiter and admin access.
- Private object storage; never expose resumes through public permanent URLs.
- File type, size and content validation plus malware scanning.
- Rate limiting, abuse protection and API request validation.
- Audit logs for sensitive actions.
- Encryption at rest where supported by the storage/database environment.
- Data retention and deletion workflows.
- Secrets stored outside source code using a secrets manager/environment configuration.

8. Bias & Evaluation Controls

The scoring pipeline should evaluate job-relevant evidence rather than irrelevant personal attributes. Gender, religion, caste, age, accent, location or college prestige should not be competence signals. For high-impact recruiting workflows, the system should provide human review and auditability rather than making an autonomous hiring decision.

9. Scalability Architecture



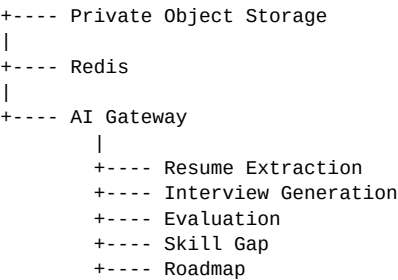
AI generation is variable-latency, so normal API requests and AI jobs should have separate performance objectives. Use asynchronous workers for resume parsing, report generation and other long-running operations. Horizontal scaling should be introduced after observing real workload and bottlenecks.

10. MVP Architecture

```

Frontend
|
FastAPI Backend
|
+---- PostgreSQL
|

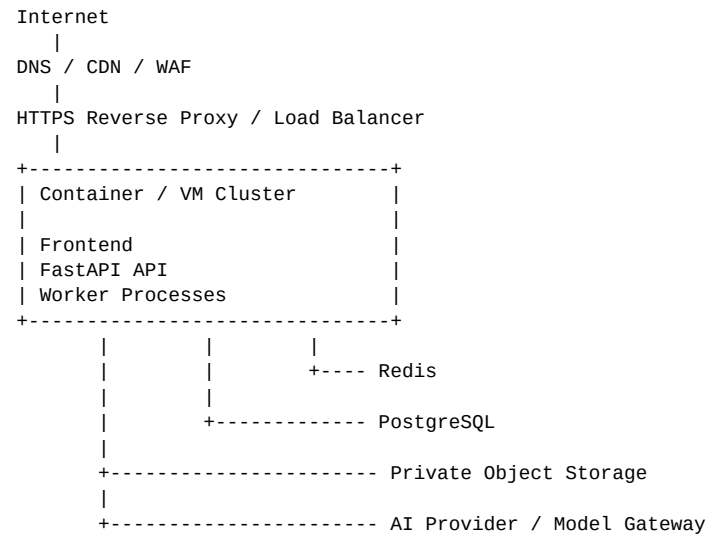
```



11. Recommended Technology Stack

Layer	Recommendation
Frontend	HTML5, CSS3, JavaScript, Bootstrap; jQuery only where useful for legacy/simple interactions.
Backend	Python + FastAPI
Database	PostgreSQL
Cache / Queue	Redis
Workers	Celery or equivalent
Storage	S3-compatible private object storage
AI	LLM behind an application-controlled AI Gateway
Voice — future	Speech-to-Text + Text-to-Speech
Deployment	Docker + reverse proxy/load balancer; cloud deployment as required
Observability	Centralised logs, metrics, tracing and AI usage/cost monitoring

12. Deployment Architecture



13. Critical Engineering Rules

- Never trust resume content as verified truth; treat it as candidate-provided evidence.
- Never make the LLM the only source of truth for deterministic tasks such as code execution or objective rule checks.
- Keep prompts, model configuration and evaluation rubrics versioned.
- Store interview state so a failed request does not destroy an active interview.
- Use idempotency for expensive background jobs.
- Keep sensitive documents private and enforce least-privilege access.
- Log enough metadata to debug AI decisions without unnecessarily storing sensitive content.

- Design the MVP modularly so individual AI components can be replaced without rewriting the whole backend.

14. Final Architecture Principle

Resume → Profile → Job Match → Interview Blueprint → Adaptive Interview → Evidence-based Evaluation → Skill Gap → Roadmap → Reassessment

This closed loop is the core of the product. The architecture should optimise for reliable evidence collection and repeatable evaluation, not merely for generating impressive AI conversation.